

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

anvl: Composable Code Transformations for Array Programs in R

Sebastian Fischer
LMU Munich, MCML

Daniel Falbel
Posit Software, PBC*

Louis J. M. Aslett
Durham University

Tomasz Kalinowski
Posit Software, PBC

Nikolai German
LMU Munich*

Martin Binder
LMU Munich, MCML

Hugh Graham
Belian

Troy Hernandez
cornball.ai

Bernd Bischl
LMU Munich, MCML

Abstract

We introduce the R package **anvl**, a code transformation framework for numerical array programs. The package allows to just-in-time (JIT) compile and differentiate R functions written against its array interface and run them on both CPUs and NVIDIA GPUs. It thus has a unique combination of features and strives to serve as a foundation for implementing numerical algorithms, such as model fitting routines, natively in R. The primary compilation path is via the OpenXLA compiler via the packages **stablehlo** and **pjrt**, but a second, experimental backend translates the R code to Fortran via **quickr**. This is possible due to **anvl**'s modular design, which separates the array interface, semantic transformations, and execution into distinct components while allowing each of them to be extended independently. The package strives to provide semantics and an interface that are familiar to R and focuses on numeric precision, e.g. in the type system and the implementations of the distribution functions. We illustrate the package's design, demonstrate its capabilities by implementing a Hamiltonian Monte Carlo sampler, and evaluate its performance against

Keywords: automatic differentiation, just-in-time compilation, array programming, GPU, XLA, R.

*Affiliation at the time the work was done.

1. Introduction

Most statistical and machine learning methods are, at their core, sequences of operations on arrays, which R can execute reasonably efficiently via vectorized operations and optimized linear algebra routines. There are, however, various limitations to this approach. First, while vectorized operations are reasonably efficient, there is still a considerable interpreter overhead and rewriting the performance-critical parts in C++ via **Rcpp** (Eddelbuettel and François 2011) or in Rust via **rextendr** (Wilke *et al.* 2026) can yield significant speed ups. Second, many algorithms require derivatives, and implementing them by hand is tedious. For example, when no gradient function is supplied via its argument `gr`, base R's `optim()` approximates the gradient by finite differences for its gradient-based methods (R Core Team 2025a). Finite differences have well-known drawbacks (Nocedal and Wright 2006, Section 8.1): each gradient costs two evaluations of the objective per parameter, and the step size has to balance the truncation error of the approximation, which grows with the step, against rounding error, which grows as the step shrinks, so that no single default suits all problems. Third, for highly parallelizable workloads, execution on graphic processing units (GPUs) can lead to orders-of-magnitude speedups.

The R package **anvl** addresses these limitations by offering a just-in-time (JIT) compiled array computing framework with support for composable code transformations and hardware acceleration. This approach has been popularized in Python by **JAX** (cite), but has since also made its way into other frameworks such as **PyTorch** (cite) via `torch.functional`. Without the ability to compile GPU programs and distribute them, e.g. via CRAN, the R ecosystem thus misses out on the large gains that come from modern accelerators which are driven by heavy industry investments. This limits both the research that can be conducted in R, as well as the implementation of such algorithms in R packages. Potential applications in the field of statistics include Markov Chain Monte Carlo (MCMC) methods, variational inference, or hidden markov models. Packages that already build on **anvl** include the choice modelling package **apollo** (Hess and Palma 2019) and the lazy raster engine **garry** (Graham 2026) for remote sensing.

At its core, **anvl** exposes three types of functions: array constructors, array operations, and code transformations, which includes semantic rewrites and compilation. A user can implement a numeric algorithm natively in R against the **anvl** API and subsequently transform it, e.g. by obtaining the gradient of a function via `gradient(f)`. Such a code transformation is similar in spirit to **base** R's `Vectorize`, but actally rewrites the underlying code, which allows for both more efficient and sophisticated transformations. The core idea of this code transformation framework is adopted from **JAX**. It consists of tracing the execution of the R program into an intermediate representation which can be rewritten by code transformation functions. Tracing involves running an R function with abstract inmputs and recording the operations instead of actually executing them. For execution, the intermediate representation is translated to a target language and subsequently compiled. The primary compilation path goes through the OpenXLA compiler (OpenXLA Team 2026), which also powers **JAX** and supports compilation to a variety of hardware backends, although **anvl** currently only supports CPUs and NVIDIA GPUs. We are interfacing the OpenXLA compiler and runtime via the companion packages **stablehlo**, which translates the intermediate representation to StableHLO (cite), and **pjrt**, which provides the JIT runtime. Through **anvl**'s backend-agnostic design that separates the what gets computed from the how it gets computed, we are also

abble to offer a second, experimental, backend that translates programs to Fortran via **quickr** (Kalinowski 2026). One design goal of **anvl** is to focus on precision, both in the design of the type system, but also in the implementation of the API. For example, the statistical distribution functions are designed with a strong focus on numerical accuracy and are rigorously benchmarked with the results available on the anvl-bench website[1]. Moreover, our goal is to offer an API that is familiar to R users and to keep the semantics of array operations consistent with base R, unless there are specific reasons to deviate.

The remainder of this article is structured as follows. Section 2 positions **anvl** among related R packages and array libraries in other languages. Section 3 introduces the OpenXLA compiler ecosystem and the two packages **stablehlo** and **pjrt**. Section 4 describes the design of **anvl** and Section 5 how to extend it. In Section 6, we implement a Hamiltonian Monte Carlo sampler, whose runtime we evaluate in Section 7. Section 8 discusses the limitations of **anvl**, and Section 9 concludes.

2. Related work

2.1. Array computing in R

Besides writing extensions in compiled languages, there are three routes to accelerated and differentiable array computing in R, whose representatives Table 1 compares with **anvl**: wrapping a Python library via **reticulate** (Kalinowski *et al.* 2026b), using **torch**, or compiling a domain-specific or restricted language.

Extensions in compiled languages. The most common way to speed up R code is to rewrite its performance-critical parts in a compiled language, either directly against R's C API (R Core Team 2025b) or in C++ via **Rcpp** (Eddelbuettel and François 2011) or in Rust via **rextendr** (Wilke *et al.* 2026), which convert R objects to and from the other language and integrate the compiled code into an R package. This removes the interpreter overhead and gives full control over the implementation, but derivatives and GPU support are left to the developer: gradients have to be derived by hand or with an AD library for the other language, such as **CppAD** in **TMB** below, and GPU code has to be written as CUDA or OpenCL kernels and built with the corresponding toolkit. Moreover, the package has to be maintained in two languages, and a compiler toolchain is needed to install it from source.

Wrappers of Python libraries. **tensorflow** (Allaire and Tang 2025), **keras3** (Kalinowski *et al.* 2026a), and **greta** (Golding 2019), a probabilistic programming language built on **TensorFlow Probability**, give access to XLA-compiled programs, and **JAX** itself can be used via **reticulate**. They inherit the maturity and large feature set of these libraries, but also the costs of the language boundary. First, every session requires a Python environment with matching package versions, and as only one Python interpreter can be initialized per R session, two R packages with conflicting Python requirements cannot be used together (Posit Software, PBC 2026b). Second, values are converted between two type systems whose semantics differ: in **reticulate** 1.47.0, a numeric `NA` and `NaN` both become `nan`, an integer `NA` becomes the plain

[1] https://r-xla.github.io/anvl-bench/

integer -2^{31}, and Python integers outside the 32-bit range can silently turn into wrong values when returned to R, e.g., `2**40` into `0` (ref 2018, 2024). Missing values are a problem beyond **reticulate**: most array libraries do not know R's conventions for them, as Appendix B shows for the native packages compared below. Third, R's garbage collector does not know how much memory a Python object holds; in our experiments, repeatedly assigning an 800 MB **NumPy** array to the same R variable increased memory usage by about 760 MB per iteration until `gc()` was called. Finally, indices and reshaping follow Python's 0-based, row-major conventions unless a wrapper translates them function by function (Posit Software, PBC 2026a,c), and errors are reported as Python exceptions.

Native array libraries. **torch** (Falbel and Luraschi 2025b) brings the **LibTorch** library that underpins **PyTorch** to R without a Python dependency and is the foundation of the deep learning ecosystem in R. Like **PyTorch**, it is designed for eager execution, in which each operation runs immediately: tracing to TorchScript is possible, but TorchScript targets deployment and is no longer developed upstream, and `torch.compile()` (Ansel *et al.* 2024) operates on Python bytecode and is not available in R. For large neural networks, where the runtime is dominated by individually expensive operations, eager execution is rarely a bottleneck, but for the comparatively small models common in statistics, the per-operation overhead of dispatching each operation from R can dominate (Section 7). Packages that give R users direct access to GPUs have had a difficult history: CUDA packages such as **gputools** (Buckner *et al.* 2010) were archived on CRAN between 2015 and 2018, **gpuR** (Determan 2019), which provided OpenCL-backed matrix classes, followed in November 2025, and **gpuMagic** (Wang and Morgan 2025), which compiled a subset of R to OpenCL, was removed from Bioconductor with release 3.23. Of the remaining packages, **OpenCL** (Urbanek 2026) requires users to write kernels in OpenCL C, and **GPUmatrix** (Lobato-Fernández *et al.* 2025) builds on **torch** or **tensorflow**; this history is one reason why **anvl** delegates hardware support to OpenXLA's PJRT plugins.

Compilers and AD for R and domain-specific languages. The byte-code **compiler** included in R (R Core Team 2025a) compiles arbitrary R code, but because it has to preserve R's full semantics, the optimizations it can perform are limited (Morandat *et al.* 2012). **quickr** (Kalinowski 2026) compiles a typed subset of R to Fortran, and **ast2ast** (Krämer and Xie 2026) translates a subset of R to C++ with AD, while **odin** (FitzJohn 2025) and **rxode2** (Fidler *et al.* 2025) compile domain-specific languages for differential equations to C. Statistical modeling languages form another group. Stan (Carpenter *et al.* 2017) compiles its probabilistic programming language to C++ with a highly optimized reverse-mode AD library (Carpenter *et al.* 2015), is available from R via **rstan**, **cmdstanr** (Gabry *et al.* 2025), and interfaces such as **brms** (Bürkner 2017), and can evaluate some functions on GPUs via OpenCL. **nimble** (de Valpine *et al.* 2017) generates C++ from a BUGS-like language and from a subset of R, and **TMB** (Kristensen *et al.* 2016) differentiates C++ templates, both via **CppAD** (Bell 2012); **RTMB** (Kristensen 2025) records **TMB**'s AD tape by evaluating R code with overloaded operators, and **dual** (Sartore 2025), **madness** (Pav 2023), **nabla** (Towell 2026), and **salad** (Perdry 2024) implement forward-mode AD via operator overloading in R. These packages are mature and highly optimized for the class of problems they target, but none of them combines JIT compilation of general R code, reverse-mode AD, and GPU execution. Most of these compilers translate R or their own language into C, C++, or Fortran and leave the optimization

to a general-purpose compiler, which only sees loops, scalar operations, and library calls, so optimizations that rely on the semantics of array operations have to be built into the code generator or a library such as **Eigen** (Guennebaud *et al.* 2010). A compiler toolchain with an intermediate representation for array programs, such as StableHLO and XLA, instead runs domain-specific optimization passes over the whole program, e.g., it simplifies algebraic expressions over arrays, chooses memory layouts, and fuses chains of elementwise operations and reductions into single kernels that avoid materializing intermediate arrays, before it generates code for the target hardware (Section 3).

2.2. Array libraries and code transformation frameworks

REVIEW: We shouldn't formulate it like this. The default language for numerical computing with accelerators is Python, with **TensorFlow** (Abadi *et al.* 2016), **PyTorch** (Paszke *et al.* 2019), and **JAX** (Bradbury *et al.* 2018). **TensorFlow** initially built a computational graph that was optimized as a whole, while **PyTorch** became popular for its eager execution; today, **TensorFlow** also executes eagerly and **PyTorch** can compile programs via `torch.compile()` (Ansel *et al.* 2024). **JAX** describes itself as a framework for composable transformations of **NumPy** programs (Harris *et al.* 2020): a function is traced into an intermediate representation, which can be differentiated in forward and reverse mode, vectorized via `vmap()`, and JIT compiled via the XLA compiler. This functional style goes back to **Autograd** (Maclaurin *et al.* 2015), the precursor of **JAX**, which differentiates native Python and **NumPy** code, including loops, branches, recursion, and closures. Its `grad()` takes a function and returns a function that computes its gradient, and reverse and forward mode can be composed, e.g., for higher-order derivatives. For this, **Autograd** wraps **NumPy**'s functions so that every operation applied to the inputs is recorded in a computational graph while the function is evaluated, and computes the gradient by traversing this graph backwards. Because the graph is recorded anew on every call, control flow needs no special constructs, but the operations are executed one at a time by **NumPy**, without compilation. **JAX** was introduced as **Autograd** and XLA brought together (Bradbury *et al.* 2018): it keeps **Autograd**'s API, but traces a function into an intermediate representation that is transformed and compiled as a whole, and extends the functional style to a set of composable transformations, such as `jvp()`, `vjp()`, `jacfwd()`, `jacrev()`, `hessian()`, and `vmap()`. **anvl** follows **JAX** in this respect: tracing produces a graph that is transformed and compiled as a whole (Section 4.3). It has since spread beyond **JAX**: **PyTorch** offers the same transformations in its `torch.func` module, which originated in **functorch** (He and Zou 2021), and Apple's **MLX** (Hannun *et al.* 2023) provides `grad()`, `vmap()`, and `compile()` in the same style. Other languages have followed similar routes. In Julia (Bezanson *et al.* 2017), **Reactant.jl** (EnzymeAD Contributors 2026) compiles Julia functions to StableHLO and runs them via XLA, **Nx** (Elixir Nx Team 2026) traces numerical definitions in Elixir and compiles them with XLA, and **GoMLX** (Pfeifer 2026a) does the same for Go via the PJRT wrapper **gopjrt** (Pfeifer 2026b). **GX** (Degris 2024) goes a step further and defines a statically typed language for array programs, in which the shapes of arrays are part of their types, builds on the backends of **GoMLX**, and provides utilities to generate bindings for Go, Python, and C++. The variety of these projects illustrates that OpenXLA's standardized interfaces (Section 3) allow a new language to obtain a compiler and hardware support without implementing them itself, which is the route that **anvl** takes for R.

2.3. Unique features of anvl

Although there are a variety of R packages that support compilation and automatic differentiation, **anvl** has some features that set it apart from the rest. Most importantly, it combines JIT compilation, automatic differentiation, GPU execution, and a native, R-like interface.

There are a few packages that almost have this combination:

- **torch**, but it's jit compilation capabilities are more limited and it uses a technology stack (TorchScript) that is no longer actively developed.
- **tensorflow** has very similar capabilities, but it uses Python and has a rather unusual interface for R users.
- Stan compiles models and computes gradients, but only its built-in functions with OpenCL implementations, such as 38 log densities, run on the GPU, while user-defined functions, e.g., a custom likelihood, and the rest of the model run on the CPU. In **anvl**, every program can run on a GPU, including user-written likelihoods and the sampler around them, because the whole program is compiled by XLA.

To the best of our knowledge, **anvl** is the first native R package that offers JIT compilation of R code, reverse-mode AD, and GPU execution at the same time. Beyond this combination, it differs from the packages above in several ways. First, a program is *staged* in R: tracing evaluates the R function and records only the array operations, so any R code can be used to generate a program, while the program itself runs without the R interpreter and is optimized as a whole by XLA. Compiled code can therefore be written like ordinary R code: functions take and return nested lists of arrays (Section 4.3), such as a list of model parameters or the state of a sampler, helper functions need no type annotations, and closures, higher-order functions such as `lapply()` or `Reduce()`, and static arguments, such as strings, flags, or lists of settings, can be used to construct the program. Compilers based on static analysis, such as **quickr**, **ast2ast**, and the run code of **nimble**, instead accept a typed subset of R whose inputs are typed scalars and arrays, and **RTMB**, which also traces R code, expects the parameters as a list of numeric arrays and cannot branch on their values. Second, compilation and differentiation are transformations of R functions that compose, rather than properties of a separate model language as in Stan or **TMB**; a gradient can be used inside a compiled sampler that is itself called from another compiled function. Third, the array interface follows R rather than Python: indexing is 1-based, arrays are laid out in column-major order, and the common R generics work on arrays, whereas wrappers of Python libraries and, to a lesser extent, **torch** follow **NumPy**'s conventions. In contrast to **quickr**, which compiles a subset of base R as it is, **anvl** provides an array library of its own that builds on the R generics but resolves inconsistencies of base R, such as the dropping of axes in subsetting (Section 4.2), and adds what base R lacks, such as further data types and random number generators with explicit state. Fourth, R values have no data type of their own and take the precision of the arrays they meet, and promotion rules are declared per function (Section 4.8), so single and double precision can be mixed deliberately, which matters on GPUs. Fifth, programs are expressed in StableHLO, a standardized intermediate representation (Section 3), which connects **anvl** to the OpenXLA ecosystem without a Python dependency: **pjrt** executes programs exported from **JAX**, such as a log density written in Python, and **JAX** can compile the StableHLO programs that **anvl** produces. The same interface opens the door to other compilers that

consume StableHLO, such as IREE (The IREE Authors 2019), which supports dynamic shapes and further hardware, without changing how programs are written in R. Sixth, the design is backend-agnostic: a traced program is an `AnvlGraph` that does not depend on how it is executed, and a backend, an `AnvlBackend` object (Section 5), lowers the graph to an executable. The same R code therefore runs via XLA on CPUs and GPUs or, with the experimental **quickr** backend, as compiled Fortran, selected via the option `anvl.backend`, and further backends can be added without changing user code. Finally, the packages are extensible from R: primitives, AD rules, lowering rules, graph transformations, and backends are R objects (Section 5), and except for the PJRT runtime, **anvl** and **stablehlo** are pure R packages that need neither Python nor a local CUDA toolkit.

2.4. Relation to JAX

JAX, from which much of **anvl**'s design is adopted, provides forward-mode AD and, combined with reverse mode, efficient Jacobians and Hessians, whereas **anvl** computes gradients of scalar functions in reverse mode, which can be nested for higher-order derivatives, and cannot differentiate through the loop primitive `nv_while()`. **JAX**'s `vmap()` vectorizes a function automatically, while batched code, such as the multi-chain sampler in Section 6, has to be written by hand in **anvl**, and **JAX** can distribute a single program over many devices, whereas **anvl** places each array on one device. **JAX** also has a large ecosystem of libraries for optimization, probabilistic programming, and neural networks. On the other hand, **anvl** follows R's conventions, broadcasts only scalars implicitly instead of following **NumPy**'s broadcasting rules (Section 4.2), never converts R data to a default precision when it enters a program (Section 4.8), and makes the graph, its transformations, and the backends accessible as R objects; and as both lower to StableHLO, **pjrt** can also execute programs exported from **JAX**.

3. Background: The OpenXLA compiler ecosystem

OpenXLA (OpenXLA Team 2026) is an open-source compiler ecosystem for machine learning that originated in **TensorFlow**. It decouples array computing frameworks (the frontends) from hardware-specific compilers and runtimes (the backends) through two standardized interfaces: StableHLO (StableHLO Team 2026), a portable intermediate representation for array programs, and PJRT (PJRT Team 2026), a runtime interface for compiling and executing such programs on a device. A framework that emits StableHLO and executes it via PJRT can run on any hardware for which a PJRT plugin exists, including CPUs, NVIDIA GPUs, and Google TPUs, which turns an $n \times m$ problem (n frontends, m types of hardware) into an $n+m$ problem. The main compiler consuming StableHLO is XLA, which, among other things, fuses operations to reduce memory traffic, but alternatives such as IREE (The IREE Authors 2019) exist.

StableHLO is a dialect of MLIR, the Multi-Level Intermediate Representation of the LLVM project, and a strongly typed, functional language in static single assignment form, in which the type of every value, i.e., its data type and shape, is known when the program is compiled. Its around 100 operations range from elementwise arithmetic to control flow, reductions, and random number generation. The **stablehlo** package, which is written in R, provides an S3 representation of the language, type inference for its operations, and a builder API. Below, we create a program that multiplies two `f32` matrices elementwise; each `hlo_*()` call checks

	anvl	**torch**	**tensorflow**, **greta**	Stan	**quickr**	**ast2ast**	**TMB**, **RTMB**	**nimble**
Compiler	XLA; Fortran (experimental)	– (eager)	XLA (optional)	C++	Fortran	C++	C++ (**TMB**); AD tape (**RTMB**)	C++
AD	rev (scalar functions), HO (nested rev)	rev, HO	fwd, rev, HO	rev (fwd and HO internal only)	–	fwd, rev (Jacobians)	rev, HO (fwd internal only)	rev, HO (up to second order)
Hardware	CPU, NVIDIA GPU	CPU, NVIDIA and Apple GPUs	CPU, NVIDIA GPU	CPU; GPU (log densities of 38 distributions, incl. GLMs, via OpenCL)	CPU	CPU	CPU	CPU
Data types	bool, i8–i64, u8–u64, f32, f64 (chosen per computation)	bool, i8–i64, u8, f8, f16, bf16, f32, f64, complex	bool, i8–i64, u8–u64, f16, bf16, f32, f64, complex (**greta**: f32, f64)	i32, f64, complex	bool, i32, f64, complex	bool, i32, f64	f64 (integers as data)	bool, i32, f64
Shapes	static, recompiled per shape	dynamic	static under XLA	dynamic	dynamic	dynamic	dynamic	dynamic
Conventions	1-based, column-major	1-based, row-major	0-based, row-major (partly translated)	1-based, column-major	1-based, column-major	1-based, column-major	0-based in C++; 1-based in R	1-based, column-major
Dependencies	PJRT plugin (downloaded)	**LibTorch** (downloaded)	Python, **TensorFlow**	C++ toolchain, CmdStan	Fortran compiler	C++20 toolchain	C++ toolchain (**TMB**)	C++ toolchain

Table 1: Comparison of **anvl** with R packages for compiled or differentiable numerical computing. Stan is used via **cmdstanr** or **rstan**. –: not available; fwd: forward mode; rev: reverse mode; HO: higher-order derivatives.

Aspect	Functionality in **anvl**
Transformations	`jit()` with caching, static arguments, and buffer donation; `gradient()` and `value_and_gradient()` for reverse-mode AD of scalar functions, which can be nested; `with_dtypes()`; all of them compose.
Backends and devices	`"pjrt"` (XLA) on CPUs and NVIDIA GPUs via PJRT plugins, with several devices per platform; experimental `"quickr"` backend for 55 of the 101 primitives (f64, i32, and booleans). Asynchronous execution.
Data types	Booleans, signed and unsigned integers of 8 to 64 bits, f32, and f64. R values take the data type of the arrays they meet; scoped default data types and per-function promotion rules (Section 4.8).
Array interface	About 190 `nv_*()` functions and methods for about 100 R generics, including arithmetic, math, and comparison operators, reductions, `[` and `[<-`, `%*%`, `c()`, `cbind()`, `sort()`, `quantile()`, and cumulative functions; 1-based and column-major (Section 4.2).
Linear algebra	`solve()`, `chol()`, `qr()`, LU, SVD, symmetric eigendecomposition, and determinants via LAPACK (CPU) and cuSOLVER (GPU).
Random numbers	Generators with explicit state for normal, uniform, and binomial variates and sampling; density, distribution, and quantile functions of four distributions.
Control flow and I/O	`nv_if()`, `nv_while()`, and `nv_scan()`, and R control flow during tracing (Section 4.3); saving and loading arrays in the safetensors format.
Extensibility	New primitives with StableHLO, reverse-mode, and **quickr** rules via `new_primitive()`; custom calls into C/C++ via **pjrt**; new backends via `AnvlBackend` (Section 5).

Table 2: Overview of the functionality of **anvl**, complementing the comparison with other packages in Table 1.

that the operation is valid for its inputs and records it in the function.

```
R> library(stablehlo)
R> func <- local_func("main")
R> x <- hlo_input("x", "f32", shape = c(2, 2))
R> y <- hlo_input("y", "f32", shape = c(2, 2))
R> func <- hlo_return(hlo_multiply(x, y))
R> func

func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
%0 = stablehlo.multiply %x, %y : tensor<2x2xf32>
return %0 : tensor<2x2xf32>
}
```

The **pjrt** package, written in C++ with **Rcpp** (Eddelbuettel and François 2011), wraps the PJRT C API, which defines how to allocate buffers on a device, compile a program, and execute it; e.g., `pjrt_compile()` compiles the textual representation `repr(func)` of the program above, and `pjrt_execute()` runs it on buffers created by `pjrt_buffer()`.

As building a PJRT plugin requires compiling LLVM, MLIR, and XLA, **pjrt** downloads prebuilt plugins when they are first needed, after asking for permission, and a companion R package installs the NVIDIA libraries required by the CUDA plugin, so users need a compatible driver but no local CUDA toolkit. Because of these downloads, **anvl**, **stablehlo**, and **pjrt**, which are MIT-licensed, are currently distributed via R-universe rather than CRAN.

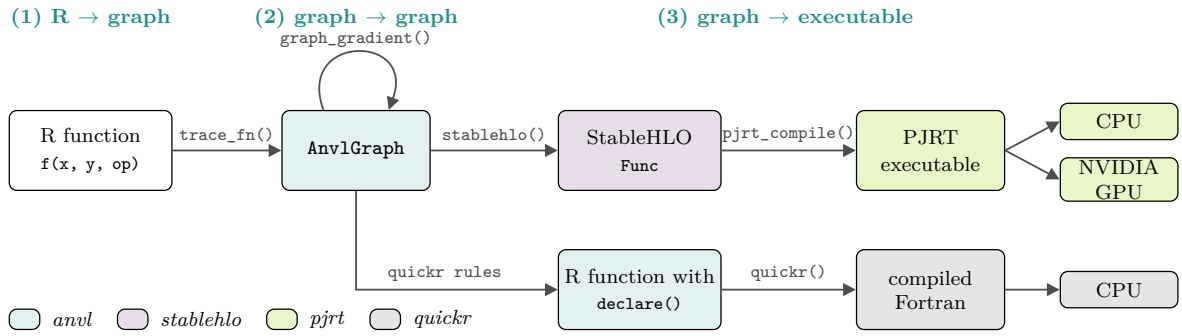


Figure 1: The three kinds of transformations in **anvl**. A node's color indicates the package that implements it. Users call `jit()`, `gradient()`, and `value_and_gradient()`, which apply these steps once the types of the inputs are known.

4. Package design

The central idea of **anvl**, adopted from **JAX**, is to view compilation and differentiation as code transformations, of which there are three kinds (Figure 1): an R function is traced into an `AnvlGraph`, a graph is transformed into another graph, e.g., by reverse-mode AD, and a graph is lowered to an executable by one of the backends. The minimalistic Python library **microjax** (The MicroJAX Authors 2024) gives a great summary of this code transformation approach.

4.1. Arrays and eager execution

The main data structure is the `AnvlArray`, which, compared to an R array, supports more data types, lives on a specific device, such as a CPU or a GPU, and represents scalars as arrays without axes. Operations are available through `nv_*()` functions and the corresponding R generics, and called on arrays, they are executed immediately, which is called *eager* execution. An `AnvlArray` holds a buffer in device memory, which an R package would usually represent as an external pointer. External pointers do not survive serialization, so objects holding them become invalid when they are saved with `saveRDS()` or sent to another R process, a common obstacle for R packages that manage foreign memory. **pjrt** therefore wraps buffers in ALTREP objects (Tierney *et al.* 2018), whose serialization methods copy the data to the host and upload it again when the object is loaded, onto the default device of the loading session. Arrays can thus be saved like any other R object and passed to parallel workers, e.g., via **parallel**, without explicit conversion. Plain R values take the data type of the array they meet, so the literal `2` below becomes an `f32` (Section 4.8).

```
R> library(anvl)
R> x <- nv_array(1:6, shape = c(2, 3), dtype = "f32")
R> sum(x * 2)

AnvlArray
 42
[ CPUf32{} ]
```

Under the hood, there are two layers of functions: *primitives*, named `prim_*()`, are the minimal operations that the transformations know about, each defined by a set of rules, and the user-facing `nv_*()` functions build on them and add broadcasting of scalars, type promotion, and default arguments. Each primitive is itself a jitted function, so eager execution compiles and caches one small program per primitive and input type; this is convenient for interactive work, but for performance, the whole program should be compiled. Execution is asynchronous: calling a compiled program returns as soon as the computation has been scheduled, and its result is only awaited when its values are needed, e.g., by `as_array()`, which allows R to prepare the next call while the device is busy. This matters because R itself is single-threaded and comparatively slow to dispatch work. On a GPU, synchronous calls would leave the device idle while R prepares the next call, whereas with asynchronous execution, R enqueues work ahead of the device and the time spent in R is hidden, as long as each call does enough work. On the CPU, the computation runs on XLA's own thread pool, so R can schedule programs on several devices from a single thread, which then run in parallel; the multi-chain benchmark in Section 7 uses this to run groups of chains on separate CPU devices. Synchronization only happens where R needs the values, e.g., in `as_array()` or `await()`, and the latter is also what benchmarks have to call to measure the computation rather than its dispatch.

4.2. An R-like array interface

Array libraries such as **NumPy**, and with it **JAX** and **PyTorch**, follow Python's conventions: 0-based indexing, row-major memory layout, and broadcasting that aligns shapes from the last axis. **anvl** instead aims to feel familiar to R users, so that numerical code written for base R carries over with few changes. Arrays have methods for about 100 R generics (Table 2), whose semantics follow base R: indexing is 1-based, subsetting uses R's bracket syntax, and arrays are laid out in column-major order, which matters whenever an operation traverses the elements of an array. For example, `cumsum()` runs over a matrix column by column in both base R and **anvl**, whereas the same computation in **torch**, which requires flattening the tensor first, follows row-major order:

```
R> m <- matrix(1:6, nrow = 2)
R> cumsum(m)

[1]  1  3  6 10 15 21

R> as_array(cumsum(nv_array(m)))

[1]  1  3  6 10 15 21

R> t <- torch::torch_flatten(torch::torch_tensor(m))
R> torch::as_array(torch::torch_cumsum(t, dim = 1))

[1]  1  4  9 11 15 21
```

Appendix A lists further operations of base R's array interface that are hard to express or give surprising results, and how **anvl** handles them. Where **anvl** needs functions of its own, their names and arguments follow base R, e.g., `nv_pmax()`, `nv_which_max()`, `nv_ifelse(test,`

`yes, no)`, `nv_seq(from, to, by)`, and `nv_rnorm(shape, state, mean, sd)`. R has no scalars, and its arrays cannot have zero axes: a number is a vector of length one, `array(1, dim = integer(0))` is an error, and a vector, which has no `dim`, is a different object from an array with one axis. Whether a value is a scalar, a vector, or a matrix is therefore not part of its type, and code has to check it at run time, e.g., via `length(x) == 1`. The missing distinction also explains why R needs a second subsetting operator for lists: as a single index is a vector of length one, `l[i]` cannot tell whether one element or a sublist of length one is requested and always returns a list, so the element itself has to be extracted with `l[[i]]`. With scalar indices, as in **anvl**, where `x[i]` with a scalar index drops the axis and an array index keeps it, a single operator could express both. In **anvl**, the shape of an array can have any length, including zero, so scalars, vectors, matrices, and arrays of higher rank are instances of one type whose number of axes is known: a reduction over all axes returns an array without axes, and the condition of `nv_if()` must be one. This distinction matters in practice: in an analysis of a large corpus of R packages, Turcotte *et al.* (2020) found that a third of all function arguments are scalars, and their proposed type language for R therefore distinguishes scalars from vectors. One deliberate departure concerns how R values correspond to arrays. A length-one R vector is a scalar, every other value must be an R array, whose `dim` becomes the shape, so each R value corresponds to exactly one shape. The same principle applies to subsetting: in base R, `m[1, ]` drops an axis unless `drop = FALSE` is given, so the type of the result changes when the number of selected rows happens to be one. In **anvl**, a scalar index drops the axis and an array index keeps it, whatever its length, which removes a discontinuity that would be particularly harmful in compiled programs, whose shapes are fixed:

```
R> x <- nv_array(matrix(1:6, 2))
R> shape(x[1, ])

[1] 3

R> shape(x[array(1), ])

[1] 1 3
```

For broadcasting, **anvl** does not adopt **NumPy**'s general broadcasting rules, which align shapes from the last axis and silently turn, e.g., the sum of arrays of shape (n) and $(n, 1)$ into an $n \times n$ matrix. Base R has no broadcasting between arrays, but recycles vectors over the column-major elements of an array, which agrees with broadcasting only in some cases. **anvl** therefore only broadcasts scalars automatically, as in `x * 2`, and general broadcasting is requested explicitly via `nv_broadcast_arrays()`, which aligns shapes from the first axis, the column-major counterpart of **NumPy**'s rule. The remaining deviations from base R follow from static shapes and from functional purity. Logical masks, as in `x[x > 6]`, work in eager execution, but inside `jit()` only if the mask is known during tracing, because the number of selected elements, and thus the shape of the result, depends on the values of the mask. A mask computed from the inputs of a jitted function is therefore rejected, and the same result can usually be obtained by masking instead of selecting, e.g., `sum(nv_ifelse(x > 0, x, 0))` instead of `sum(x[x > 0])`. Random number generators take and return an explicit state, because a compiled program cannot modify hidden global state such as R's `.Random.seed`: a call to base R's `rnorm()` inside a jitted function would be evaluated only once during tracing,

and its values would be constants of the compiled program.

4.3. Tracing R code into a graph

Instead of statically analyzing the source code of a function, as **quickr** does, **anvl** *traces* it: `trace_fn()` evaluates an R function with `GraphBox` objects in place of arrays, which carry only a data type and a shape. Below, we trace a function that adds or multiplies two `f32` scalars, created by `nv_aval()`, depending on an argument `op`.

```
R> f <- function(x, y, op) {
+    if (op == "add") x + y else x * y
+  }
R> s <- nv_aval("f32", integer())
R> graph <- trace_fn(f, list(x = s, y = s, op = "mul"))
R> graph

<AnvlGraph> (%x1: f32[], %x2: f32[]) {
  %1: f32[] = mul(%x1, %x2)
  return %1
}
```

Every call to a primitive appends a `GraphStatement`, consisting of the primitive, its inputs, its static parameters, and its outputs, to the graph, while all other R code is simply evaluated; here, only the branch of the `if` statement that was taken becomes part of the `AnvlGraph`. Inputs and outputs are *RTrees*, an R analog of **JAX**'s pytrees (Bradbury *et al.* 2018): arbitrarily nested lists, whose leaves are arrays or R values. Before tracing, a tree is flattened into its leaves and its nesting structure is stored separately, and after execution, the outputs are restored into their tree, so transformed functions can take and return, e.g., a named list of model parameters, and `gradient()` returns the gradient in the same structure.

Compared to static analysis, tracing allows arbitrary R code to construct a program and only needs to know the semantics of the array operations, not those of the R language as a whole. The downside is that tracing only works for functions that can be intercepted: `sum()` is a generic with a method for **anvl**'s arrays, whereas `rowSums()` is not generic and cannot be traced. Furthermore, programs must be pure: side effects, such as printing, only happen once during tracing, and R control flow can only depend on values that are known during tracing, such as static arguments and shapes.

4.4. Automatic differentiation

AD computes derivatives of a program by applying the chain rule to the elementary operations it executes, whose derivatives are known (Griewank and Walther 2008). *Forward mode* propagates derivatives from the inputs to the outputs alongside the evaluation, computing a Jacobian-vector product per pass, and is efficient for functions with few inputs. *Reverse mode* propagates the derivatives of the outputs with respect to each intermediate value, the *adjoints*, backwards to the inputs, computing a vector-Jacobian product per pass, and thus yields the gradient of a scalar function with many parameters, such as a log-likelihood, at a small constant multiple of the cost of evaluating it. Combining both, e.g., forward over reverse, gives Hessian-vector products and full Hessians efficiently. **anvl** currently implements

reverse mode.

4.5. Transforming and lowering graphs

Once a program is represented as an `AnvlGraph`, it can be transformed into another graph. Currently, the main such transformation is reverse-mode AD, e.g., `graph_gradient(graph, wrt = c("x", "y"))`. For this, every primitive stores a `"reverse"` rule, which receives the inputs and outputs of the forward call and the adjoints of the outputs, and returns the vector-Jacobian products for the inputs. `graph_gradient()` walks the graph in reverse order, applies these rules, and accumulates the adjoints of values that are used multiple times. Because the reverse rules themselves call primitives, the backward pass is recorded with the same tracing machinery, and the result is again an `AnvlGraph`.

A backend then lowers a graph into an executable. For the `"pjrt"` backend, `stablehlo()` translates the graph into a StableHLO function by applying each primitive's `"stablehlo"` rule, which maps it to one or more **stablehlo** operations and converts R's 1-based indices to StableHLO's 0-based ones. R literals are embedded into the program as constants, while arrays that the traced function closes over become additional inputs, which avoids copying their data into the program. The result is compiled via **pjrt**. The **quickr** backend instead translates the graph into an R function with type declarations and compiles it with **quickr**; compared to XLA, a compiled **quickr** function can be reused for inputs of different shapes, but it only runs on the CPU and supports fewer data types.

4.6. The user interface

Users rarely call these transformations themselves, but use `jit()`, `gradient()`, and `value_and_gradient()`, which return functions that apply them lazily, once the types of the inputs are known. `jit()` returns a function that traces, lowers, and compiles its input when it is called, and caches the executable, so that a later call with inputs of the same data types and shapes reuses it. Below, the side effect shows that the second call does not trace the function again.

```
R> f_jit <- jit(function(x, y) {
+    cat("tracing\n")
+    x * y
+  })
R> y1 <- f_jit(x, x)

tracing

R> y2 <- f_jit(x, x + 1L)
```

Arguments that are not arrays, such as `op` above, must be marked as `static`, e.g., `jit(f, static = "op")`; their values become part of the cache key, and the function is traced again for every new value. A jitted function called from within another jitted function is traced as part of the outer function, so that both are compiled into a single executable. `gradient()` takes a function with scalar output and returns a function that computes the gradient with respect to the arguments given in `wrt`, which can itself be called within a jitted function (Section 6).

R's control flow can be used within jitted functions, but an `if` statement only retains the branch that was taken and a loop is unrolled during tracing. For control flow that depends on array values, or for loops that should not be unrolled, **anvl** provides the higher-order primitives `nv_if()`, `nv_while()`, and `nv_scan()`, which trace the functions passed to them into sub-graphs. For example, `nv_scan()` runs a loop for a fixed number of steps, threads a state through a body function, and stacks the outputs of all steps; we use it in Section 6.

The main restriction stems from XLA's requirement that all shapes are known when a program is compiled: each new combination of input shapes triggers a new compilation, and operations whose output shape depends on data, such as `unique()`, cannot be expressed directly (Section 8). Moreover, StableHLO programs cannot raise errors at runtime, so **anvl** performs all checks during tracing.

4.7. Device memory

R reclaims memory with a generational mark-and-sweep garbage collector, which runs when R's own heap fills up; objects carry reference counts, but these only decide whether an object can be modified in place, and memory is reclaimed by the collector alone. In contrast to Python, whose reference counting frees an object as soon as its last reference disappears, an object that goes out of scope in R is only freed the next time the collector runs. An array on a GPU is represented in R by a small object that refers to device memory, so a loop that creates many such arrays barely grows R's heap, the collector does not run, and device memory fills up with arrays that are no longer referenced. This memory is only released by a collection, which marks all reachable R objects and blocks R in the meantime, so computations on the GPU end up waiting for R's collector. Packages that manage device memory from R therefore need a strategy of their own. **torch** combines **PyTorch**'s caching allocator with a hook that runs R's collector when an allocation fails or the reserved memory exceeds a threshold (Falbel and Luraschi 2025a). **tensorflow** and **greta** rely on Python's reference counting, which can only free a tensor once R's collector has released its reference, and **TensorFlow** reserves most of the GPU memory for its own allocator by default. Stan allocates OpenCL buffers within its C++ code and frees them deterministically, so no device memory is visible to R. For **anvl**, this is much less of a problem than for packages that execute every operation eagerly, such as **torch**, in which every intermediate result is an R object that holds on to its device memory until the collector runs. In a jitted function, intermediate results never become R objects, but are buffers that XLA allocates, reuses, and frees, and inputs can be donated via the `donate` option of `jit()`, so that their memory is reused for the outputs. Only the inputs and outputs of compiled programs and the results of eager operations are R objects, and when an allocation fails, **pjrt** runs R's collector and retries, similar to **torch**. On the CPU, arrays, including the results of compiled programs, are stored in memory allocated by R, so its collector accounts for them and runs when they fill up its heap.

4.8. Data types and precision

Numerical precision is a recurring concern in statistics: log-likelihoods sum many terms, linear systems can be ill-conditioned, and MCMC methods compare densities whose difference is small relative to their size. Accelerators, on the other hand, are designed for single precision, and consumer GPUs often execute double-precision operations at a small fraction of the single-precision throughput. A framework for statistical computing should therefore make it

easy to choose the precision of a computation, and it should not lose precision without being asked to.

R values have no data type

In **JAX**, Python scalars are *weakly typed*: they take the data type of the array they are combined with (Bradbury *et al.* 2018). **NumPy** arrays, however, have a data type, and by default, **JAX** truncates double-precision data to single precision when it enters a program, while with 64-bit mode enabled, the data promotes every single-precision array it meets. Transferred to R, where every real-valued vector is stored in double precision, either all data loses precision when it enters a program, or it pulls every computation into double precision.

anvl therefore extends weak typing to all R values that enter a program, i.e., length-one vectors as well as matrices and arrays. An R value has no data type and only *materializes* as an array when it meets one, taking its data type. Below, the same R matrix `B` is multiplied with a single- and a double-precision array.

```
R> B <- matrix(c(0.1, 0.2, 0.3, 0.4), 2, 2)
R> matmul <- jit(function(x, b) x %*% b)
R> c(dtype(matmul(nv_array(diag(2), dtype = "f32"), B)),
+    dtype(matmul(nv_array(diag(2), dtype = "f64"), B)))
```

```
[1] "f32" "f64"
```

In the second call, `B` enters the program in double precision and the product is exactly `B`, whereas a fixed default precision would either have lost precision or turned the first result into `f64`. Only a value that meets no array, such as the argument of `nv_exp(1)`, settles on a default data type. A non-static R argument of a jitted function, such as `b` above, enters the trace with a shape but no data type, and each operation that uses it decides at which data type it materializes; if it is used at several data types, it is transferred at the narrowest one that holds all of them.

The same principle guards against silent changes of values elsewhere. An R value is built directly at the data type it takes, so a value that the data type cannot represent is an error: adding `-2L` to an unsigned 8-bit integer fails instead of wrapping around. As in R, `x - 1L` keeps an integer array `x` an integer, while `x - 1` gives a float. Finally, integers never become floats on their own: an unsigned and a signed 64-bit integer have no common data type and must be converted explicitly, whereas **JAX** and **NumPy** silently promote them to a float.

Choosing the precision

The default data types for R doubles and integers are a property of the backend (`f32` and `i32` for XLA) and can be changed for a scope via `with_default_dtypes()` or the option `anvl.default_dtypes`. `with_dtypes()` turns a function into one that runs at the given data types, converting its array arguments and results and setting the defaults for its body.

```
R> loss <- function(beta, X, y) mean((X %*% beta - y)^2)
R> loss_f64 <- with_dtypes(loss, c(float = "f64"))
R> c(dtype(nv_exp(1)), dtype(with_default_dtypes(c(float = "f64"), nv_exp(1))),
+    dtype(loss_f64(nv_array(matrix(0, 2, 2), dtype = "f32"), B, B)))
```

```
[1] "f32" "f64" "f64"
```

Compiled programs are cached per default data types, so both variants can be used side by side, and in contrast to **JAX**'s 64-bit mode, the defaults only decide what an R value settles on when nothing else determines its data type.

Promotion rules

When a function receives arrays of different data types, it has to decide on the data type of the computation. In **anvl**, this is not a global rule, but each `nv_*()` function passes a *promotion rule* to `as_anvl_arrays()`, which converts its arguments to arrays at the beginning of the function. `promotion_common()` brings all inputs to their common data type, as used, e.g., by `nv_add()`; `promotion_like()` brings them to the data type of one designated input; `promotion_dtype()` to a fixed data type; and `promotion_rdata_common()`, which is used by primitives, only lets R values adapt and requires arrays to agree. The rules refuse conversions that lose precision unless `coerce = TRUE`. For example, the following function computes $ax+y$ at the data type of `y`, and refuses an `x` of higher precision rather than silently rounding it.

```
R> axpy <- jit(function(a, x, y) {
+    args <- as_anvl_arrays(a = a, x = x, y = y, .promote = promotion_like("y"))
+    args$a * args$x + args$y
+  })
R> y32 <- nv_array(c(1, 1), dtype = "f32")
R> as_array(axpy(2, nv_array(c(1, 2), dtype = "f32"), y32))

[1] 3 5

R> axpy(2, nv_array(c(1, 2), dtype = "f64"), y32)

Error:
! Cannot bring `x` to data type "f32".
x "f64" is not promotable to "f32".
i Convert it explicitly with `nv_convert()`.
```

Developers of packages built on **anvl** can thus decide for each function which argument determines the precision, e.g., that a likelihood is accumulated in double precision while the data stays in single precision.

4.9. Other features

Beyond the transformations, **anvl** provides several practical features, some of which are described in more detail above.

- *No CUDA toolkit*: `install_anvl()` downloads the PJRT plugins and, on a machine with an NVIDIA GPU, installs the CUDA libraries as the R package **pjrt.cuda**, so a compatible NVIDIA driver is all that is needed.
- *Serializable arrays*: arrays can be saved with `saveRDS()` and passed to parallel workers like any other R object (Section 4.1), and `nv_save()` and `nv_read()` write and read files in the safetensors format.
- *All CPU cores*: XLA parallelizes operations over all cores of the CPU without any configuration (Section 7).

- *Asynchronous execution*: R continues while the device computes (Section 4.1).
- *Nested lists of arrays*: jitted functions and gradients take and return nested lists of arrays, such as a named list of model parameters (Section 4.3).
- *An alternative backend*: the experimental **quickr** backend translates programs to Fortran (Section 4.5).
- *Written in R*: except for the PJRT runtime, **anvl** and **stablehlo** are written in R, which makes them easy to read, extend, and contribute to (Section 5).
- *Error messages*: errors say what went wrong and how to fix it, as in the promotion example in Section 4.8.

5. Extending the package

Most operations can be expressed in terms of existing primitives and are best added as `nv_*()` functions. A new primitive is needed if an operation cannot be expressed otherwise, or if it benefits from a dedicated implementation, e.g., of its derivative. Below, we add a primitive for the softplus function $\log(1+e^x)$, which appears in the logistic regression example in Section 6. A primitive is created via `new_primitive()`, whose body records a statement in the graph via `graph_desc_add()`, together with an inference function that computes the types of the outputs from those of the inputs and validates them. Within the body, `self` refers to the primitive being defined.

```
R> prim_softplus <- new_primitive("softplus", function(x) {
+    infer <- function(x) {
+      if (!(dtype(x) %in% c("f32", "f64"))) stop("`x` must be a float.")
+      list(AbstractArray(dtype = dtype(x), shape = shape(x)))
+    }
+    graph_desc_add(self, list(x = x), infer_fn = infer)[[1L]]
+  })
```

Next, we attach the StableHLO rule, which receives and returns **stablehlo** values, and a reverse rule, which uses the derivative $\sigma(x) = 1/(1+e^{-x})$.

```
R> prim_softplus[["stablehlo"]] <- function(x) {
+    list(hlo_log_plus_one(hlo_exponential(x)))
+  }
R> prim_softplus[["reverse"]] <- rule_reverse(
+    function(inputs, outputs, grads, params, required) {
+      list(prim_mul(grads[[1L]], prim_plogis(inputs[[1L]])))
+    }
+  )
```

The primitive can now be used eagerly, within jitted functions, and be differentiated.

```
R> grad_softplus <- jit(gradient(function(z) sum(prim_softplus(z))))
R> as_array(grad_softplus(nv_array(c(-1, 0, 1)))$z)

[1] 0.2689414 0.5000000 0.7310586
```

Operations that cannot be expressed in StableHLO can use PJRT's custom call mechanism: a handler written in C or C++ against XLA's foreign function interface is registered from R via **pjrt**, with separate implementations per platform, as the linear algebra primitives do to call LAPACK and cuSOLVER.

Since an `AnvlGraph` is a simple data structure, new graph-to-graph transformations, such as forward-mode AD, can be implemented in R by iterating over its statements and emitting new ones, guided by a rule stored in each primitive. A backend is an `AnvlBackend` object, a collection of functions that define how arrays are stored, converted to R, and placed on devices, and how a graph is compiled, together with a lowering rule for each supported primitive; exactly one backend is active at a time. We test every primitive against **torch** as a reference implementation wherever possible, comparing the outputs and gradients of jitted primitives with those of **torch** and its autograd engine on random inputs.

6. Use case: Hamiltonian Monte Carlo

Hamiltonian Monte Carlo (HMC) (Neal 2011) is an MCMC method that uses the gradient of the log posterior density to propose distant states that are still accepted with high probability, and thus a natural use case for **anvl**: it needs gradients, its loops are slow in R, and running many chains in parallel is a good fit for GPUs. We consider Bayesian logistic regression with a binary response $y \in \{0,1\}^n$, features $X \in \mathbb{R}^{n \times p}$, $y_i \sim \text{Bernoulli}(\sigma(x_i^\top \beta))$, and a standard normal prior $\beta \sim \mathcal{N}(0, I_p)$. HMC simulates the dynamics of a particle with position β and momentum $\rho \sim \mathcal{N}(0, I_p)$ in the potential energy

$$U(\beta) = -\log p(\beta \mid X, y) = -\sum_{i=1}^{n} \left(y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta}) \right) + \frac{1}{2}\|\beta\|^2 + \text{const.}$$

We write the sampler for K chains at once, so the positions are stored in a $K \times p$ matrix, and the potential returns a vector of length K.

```
R> potential <- function(beta, X, y) {
+    eta <- X %*% t(beta)
+    y <- nv_broadcast_to(y, shape(eta))
+    loglik <- nv_sum(y * eta - nv_log1p(exp(eta)), axes = 1L)
+    logprior <- -nv_sum(beta^2, axes = 2L) / 2
+    -(loglik + logprior)
+  }
```

Because the chains are independent, the gradient of the summed potential contains the gradient of each chain in the corresponding row, so a single call to `gradient()` yields the gradients of all chains.

```
R> grad_potential <- gradient(
+    function(beta, X, y) sum(potential(beta, X, y)),
+    wrt = "beta"
+  )
```

The dynamics are simulated with the leapfrog integrator, which alternates half steps of size $\varepsilon/2$ for the momentum with full steps of size ε for the position. We run L such steps with `nv_scan()`, where the gradient at the current position is part of the carry, so that each step evaluates the gradient only once.

```
R> leapfrog <- function(beta, p, X, y, eps, L) {
+    init <- list(beta = beta, p = p, grad = grad_potential(beta, X, y)$beta)
+    nv_scan(init, steps = L, body = function(s, x) {
+      p <- s$p - eps / 2 * s$grad
+      beta <- s$beta + eps * p
+      grad <- grad_potential(beta, X, y)$beta
+      p <- p - eps / 2 * grad
+      list(carry = list(beta = beta, p = p, grad = grad), out = NULL)
+    })$carry
+  }
```

A transition draws a new momentum, simulates a trajectory, and accepts the proposal of each chain with probability $\min(1, \exp(H_0 - H_1))$, where $H = U(\beta) + \|\rho\|^2/2$ is the total energy. The random number generator state is passed explicitly, and the acceptance decision uses `nv_ifelse()` instead of an R `if` statement, as it depends on array values.

```
R> hmc_step <- function(beta, state, X, y, eps, L) {
+    draw <- nv_rnorm(shape(beta), state)
+    p <- draw$values
+    prop <- leapfrog(beta, p, X, y, eps, L)
+    H0 <- potential(beta, X, y) + nv_sum(p^2, axes = 2L) / 2
+    H1 <- potential(prop$beta, X, y) + nv_sum(prop$p^2, axes = 2L) / 2
+    u <- nv_runif(shape(beta)[1L], draw$state)
+    accept <- log(u$values) < H0 - H1
+    accept <- nv_broadcast_to(accept, shape(beta))
+    list(beta = nv_ifelse(accept, prop$beta, beta), state = u$state)
+  }
```

Finally, we collect `n_draws` draws with another `nv_scan()`, whose outputs are stacked along a new first axis, and compile the whole sampler with `jit()`. The number of leapfrog steps and draws are static arguments, as they determine the structure of the program.

```
R> hmc <- jit(function(beta, state, X, y, eps, L, n_draws) {
+    out <- nv_scan(list(beta = beta, state = state), steps = n_draws,
+      body = function(s, x) {
+        s <- hmc_step(s$beta, s$state, X, y, eps, L)
+        list(carry = s, out = s$beta)
+      }
+    )
+    list(draws = out$out, beta = out$carry$beta, state = out$carry$state)
+  }, static = c("L", "n_draws"))
```

We apply the sampler to simulated data with $n = 1000$ observations and $p = 5$ features, running four chains in double precision. The posterior means, computed from the second half

of each chain, agree with the maximum likelihood estimates up to the shrinkage induced by the prior.

```
R> options(anvl.default_dtypes = c(float = "f64", int = "i32"))
R> set.seed(1)
R> n <- 1000; p <- 5
R> X <- matrix(rnorm(n * p), n, p)
R> y <- rbinom(n, 1, plogis(X %*% rnorm(p, sd = 0.5)))
R> res <- hmc(nv_array(matrix(0, 4, p)), nv_rng_state(42L), nv_array(X),
+    nv_array(as.double(y)), eps = 0.5 / sqrt(n), L = 10L, n_draws = 2000L)
R> draws <- as_array(res$draws)
R> rbind(hmc = colMeans(apply(draws[1001:2000, , ], 3, c)),
+    glm = coef(glm(y ~ X - 1, family = binomial)))

            X1        X2         X3        X4        X5
hmc -0.7730560 0.3412134 -0.8296505 0.6687511 0.5593170
glm -0.7738017 0.3425455 -0.8321399 0.6701697 0.5599573
```

The whole sampler, including the 2000×11 gradient evaluations per chain, runs as a single compiled program without returning to R. The same code runs on a GPU when the arrays are allocated there, e.g., via `nv_array(X, device = "cuda")`.

7. Runtime evaluation

We compare the runtime of the HMC sampler from Section 6 to Stan, whose AD library and code generation are the state of the art for statistical models on the CPU (Carpenter *et al.* 2015), to **TMB**, and to the same sampler written with **torch**.

7.1. Setup

We run Stan's static HMC sampler (without the No-U-Turn extension) via CmdStan 2.40 with the same step size $\varepsilon = 0.5/\sqrt{n}$, $L = 10$ leapfrog steps, the identity metric, and adaptation disabled, so that all samplers perform the same number of gradient evaluations. The Stan model uses `bernoulli_logit_glm()`, Stan's most efficient implementation of the logistic regression likelihood, which can also be evaluated on a GPU via OpenCL; on the GPU, Stan's chains run as separate processes that share the device, at most 8 at a time. **TMB** evaluates the log posterior and its gradient with **CppAD** from a C++ template, and as it does not provide an HMC sampler, we run the same static HMC loop in R around its compiled objective and gradient, as is common when custom algorithms are implemented with **TMB**. The **torch** sampler mirrors the **anvl** code, but runs the leapfrog and sampling loops in R, as **torch** executes each operation eagerly. All samplers work in double precision and start at $\beta = 0$, and the data are simulated with standard normal features and $\beta \sim \mathcal{N}(0, 0.25 I_p)$. We exclude process start-up and data loading from Stan's measurements by subtracting the time of an otherwise identical run without draws, and time **anvl** after the data has been transferred to the device. Each configuration is repeated three times after a first call that includes compilation, and we report the median.

We run three experiments. First, we draw 1000 samples from a single chain for $p = 20$ and

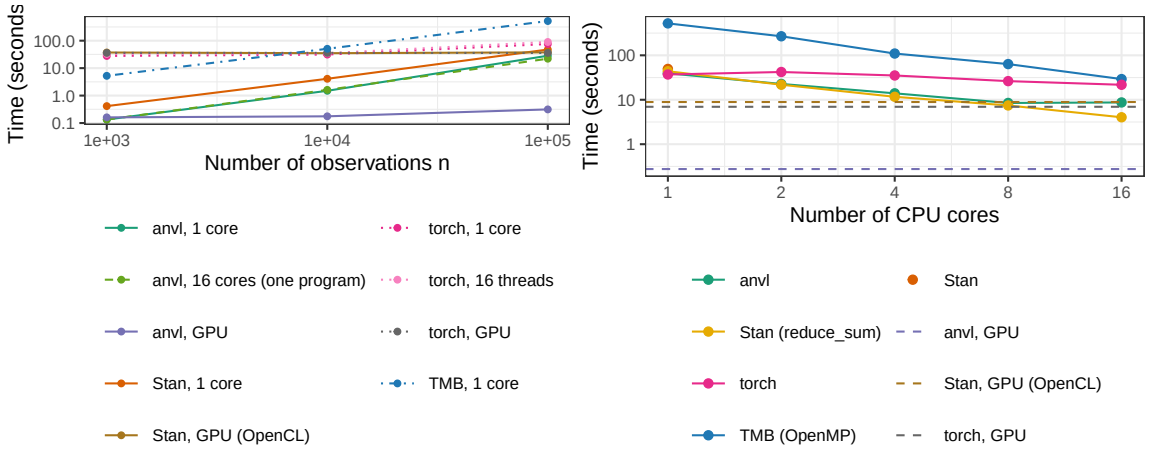


Figure 2: Left: time to draw 1000 samples from a single chain as a function of the number of observations n ($p = 20$). Right: strong scaling, time to draw 200 samples from a single chain for $n = 10^5$ and $p = 100$ on 1 to 16 CPU cores; Stan uses `reduce_sum()`, the single point is the unmodified Stan model, and the dashed lines are **anvl**, **torch**, and Stan (OpenCL) on the GPU. Medians of three repetitions, log-log scales.

$n \in \{10^3, 10^4, 10^5\}$. Second, we measure strong scaling: we draw 200 samples from a single chain for $n = 10^5$ and $p = 100$ on 1 to 16 cores. **anvl** and **torch** run the unchanged sampler, which they parallelize within operations, **TMB** evaluates the likelihood with several OpenMP threads, and Stan needs a modified model, in which `reduce_sum()` splits the likelihood into slices that are evaluated by several threads. Third, we draw 1000 samples from each of K chains for $n = 10^4$ and $p = 20$. CmdStan and **TMB** run one chain per process, at most one per core, and **anvl** runs all chains as one batched program, except on 16 cores, where we split them into 16 groups that run on separate PJRT CPU devices. All experiments ran on the LRZ AI Systems, on two Intel Xeon Gold 6336Y processors restricted to 16 physical cores and on an NVIDIA A100 80GB GPU, in a **Docker** image that pins all software versions; the image, the benchmark scripts, and instructions are in the `paper/` directory of https://github.com/r-xla/anvl.

7.2. Results

Figure 2 (left) shows the results for a single chain. On one core, **anvl** draws the samples 3.1, 2.7, and 1.7 times faster than CmdStan for $n = 10^3$, 10^4, and 10^5, respectively. A plausible reason is that XLA compiles the whole sampler into a single optimized program, whereas Stan and **TMB**, which takes 11 to 13 times as long as CmdStan, evaluate the gradient with operator-overloading AD. The GPU does not pay off for small data, but for $n = 10^5$, it draws the samples in 0.31 seconds, about 90 times faster than **anvl** on one core and 150 times faster than CmdStan. The **torch** sampler needs about 28 seconds for $n = 10^3$ on one core, 200 times as long as **anvl**, and about 36 seconds for every n on the GPU: its runtime is dominated by the overhead of dispatching each operation from R, which is the per-operation overhead of eager execution discussed in Section 2.1. Stan's OpenCL backend, which evaluates only the likelihood and its gradient on the GPU, needs 35 to 38 seconds for every n, as each gradient evaluation transfers data and launches kernels.

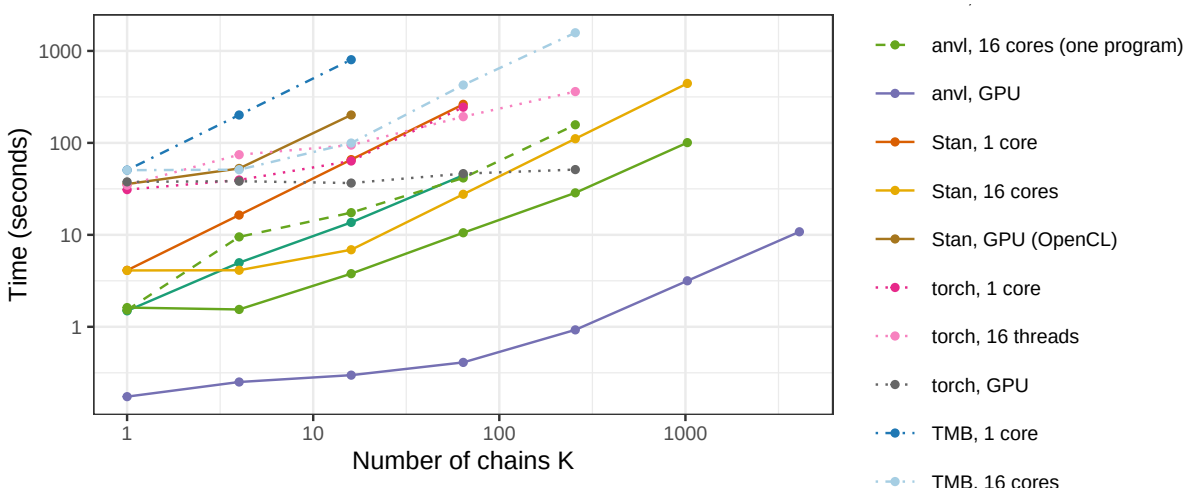


Figure 3: Time to draw 1000 samples from each of K chains for $n = 10^4$ (median of three repetitions, log-log scale).

Figure 2 (right) shows how a single chain scales with the number of cores. Without any change to the code, **anvl** becomes 1.7, 2.8, and 4.6 times faster on 2, 4, and 8 cores, and does not improve further on 16 cores, where the individual operations are too small at this problem size to keep all cores busy. With `reduce_sum()`, Stan scales further (11 times faster on 16 threads), but this requires rewriting the model, which is then slower on a single core (44 versus 50 seconds, compared to 39 seconds for **anvl**). **TMB** becomes 18 times faster on 16 threads, but starts from 522 seconds, and **torch** is as fast as **anvl** on one core, where the operations are large enough to hide its overhead, but becomes only 1.7 times faster on 16 threads. On the GPU, the same run takes 0.28 seconds with **anvl**, 7 seconds with **torch**, and 8.9 seconds with Stan's OpenCL backend.

Figure 3 shows the results for multiple chains. On one core, **anvl** is 2.8 (one chain) to 5.9 (64 chains) times faster than CmdStan, as batching turns the matrix-vector products of the individual chains into matrix-matrix products. **torch** benefits from batching as well, but is still 5.5 times slower than **anvl** for 64 chains on one core, and **TMB** takes 55 to 59 times as long as **anvl**. On 16 cores, **anvl** is 2.6 times faster than CmdStan for 64 chains and 4.4 times faster for 1024 chains; running all chains as a single program instead of splitting them over devices is considerably slower, because each operation is again too small to be distributed efficiently. On the GPU, the time of **anvl** is almost constant up to 64 chains and grows sublinearly beyond: 4096 chains with 1000 draws each take 10.8 seconds, about 380,000 draws per second, compared to 2,300 for CmdStan on 16 cores. This is the regime in which the many-chain MCMC methods discussed in the introduction operate; on the GPU, **torch** did not finish 1024 chains and Stan's OpenCL backend did not finish 64 chains within one hour. All average acceptance rates were between 0.97 and 0.99.

Finally, the one-off costs are small: for $n = 10^4$ on 16 cores, tracing the sampler, lowering it to StableHLO, and compiling it with XLA take 0.28, 0.21, and 0.24 seconds, respectively, compared to about 11 seconds for compiling the Stan model and 35 seconds for the **TMB** template.

8. Limitations

Most limitations of **anvl** follow from XLA's requirement that all shapes are known when a program is compiled, and from the transformations that **anvl** does not implement yet.

Static shapes. Because XLA compiles a program for fixed shapes, `jit()` traces and compiles a function anew for every combination of input shapes it is called with (Section 4.6). For the HMC sampler, tracing, lowering, and compilation together take about 0.7 seconds (Section 7), which is negligible when a function is called many times with the same shapes, as in a sampler or an optimizer, but adds up when the shapes change from call to call, e.g., when the same model is fitted to many data sets of different sizes. Operations whose output shape depends on the values of their inputs cannot be part of a compiled program at all: selecting elements with a logical mask that is computed inside the program, as in `x[x > 0]` (Section 4.2), or `unique()`. For the same reason, the state of `nv_while()` and `nv_scan()` keeps its shape across iterations, so an algorithm that collects a varying number of values has to preallocate an array of the maximal size. Such programs can usually be rewritten: masking replaces selection, e.g., `sum(nv_ifelse(x > 0, x, 0))` instead of `sum(x[x > 0])`, padding inputs to a few fixed sizes bounds the number of compilations, and steps whose output shape depends on the data can run eagerly or in plain R between calls of jitted functions. The **quickr** backend, whose compiled functions can be reused for inputs of different shapes (Section 4.5), and the IREE compiler (The IREE Authors 2019), which also consumes StableHLO and supports dynamic shapes, are two routes towards lifting this restriction.

Automatic differentiation. **anvl** implements only reverse mode (Section 4.4), and `gradient()` only differentiates functions with a scalar output. Jacobians and Hessians therefore have to be assembled from one reverse pass per output, whereas **JAX** combines forward and reverse mode to compute them more efficiently. Reverse mode differentiates through `nv_if()` and `nv_scan()`, but not through `nv_while()`: the number of iterations of a while loop is only known at runtime, so there is no static size for the intermediate values of all iterations, which the backward pass needs. Loops with a fixed number of iterations should therefore be written with `nv_scan()`.

Vectorization and multiple devices. **anvl** has no vectorizing transformation like **JAX**'s `vmap()`, so batched code, such as the multi-chain sampler in Section 6, has to be written by hand. Each array resides on a single device, and a program runs on the device of its inputs. Several devices can be used by running separate programs on them, as in the multi-chain benchmark in Section 7, but a single program cannot be distributed over several devices, as **JAX** does by sharding arrays.

Hardware and data types. Although XLA also targets further accelerators, **anvl** currently supports only CPUs and NVIDIA GPUs, and the **quickr** backend is experimental, runs only on the CPU, and supports fewer data types. Floating-point computations are supported in single and double precision (Table 2); half-precision types, which are common in deep learning, and complex numbers are not supported.

9. Conclusion

We introduced **anvl**, which brings composable code transformations for array programs to R: programs are traced into a graph, which can be differentiated in reverse mode and compiled via OpenXLA or **quickr**, so the same R code runs on CPUs and GPUs, with an R-like array interface and control over numerical precision. The HMC use case showed that a whole sampler can be compiled into a single program that is faster than Stan on the CPU and scales to thousands of chains on a GPU.

Its main limitations are XLA's static shapes and the restriction of AD to reverse mode (Section 8). Future work will add forward-mode AD, which, combined with reverse mode, yields Jacobians and Hessians efficiently, e.g., for Laplace approximations, and a vectorizing transformation similar to **JAX**'s `vmap()`, which would remove the need to write batched code by hand. Beyond the core, we hope that **anvl** will serve as a foundation for packages that implement statistical and machine learning algorithms, such as samplers and variational inference, on top of it. A first such package is **garry** (Graham 2026), a lazy raster engine for remote sensing that compiles the numerical kernels of a raster computation with **anvl** and executes them chunked across processes.

Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the National Research Data Infrastructure - NFDI 29/2 - project number 460135501 We thank the developers of **JAX**, whose design and documentation, in particular the Autodidax tutorial (The JAX Authors 2024), inspired many aspects of **anvl**, as well as the authors of the Go library **gopjrt** (Pfeifer 2026b), which informed the design of **pjrt**. The computations were run using resources provided by the Leibniz Supercomputing Centre (LRZ).

References

(2018). "What Should NA Be Converted to in Python? Issue #197, rstudio/reticulate." https://github.com/rstudio/reticulate/issues/197. Accessed 2026-09-24.

(2024). "Reticulate Can't Handle Large Integer Values. Issue #1647, rstudio/reticulate." https://github.com/rstudio/reticulate/issues/1647. Accessed 2026-09-24.

Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, *et al.* (2016). "TensorFlow: A System for Large-Scale Machine Learning." In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pp. 265–283.

Allaire J, Tang Y (2025). *tensorflow: R Interface to TensorFlow*. R package version 2.20.0, URL https://CRAN.R-project.org/package=tensorflow.

Ansel J, Yang E, He H, Gimelshein N, Jain A, Voznesensky M, *et al.* (2024). "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph

Compilation." In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 929–947. doi:10.1145/3620665.3640366.

Bell BM (2012). "CppAD: a package for C++ algorithmic differentiation." *Computational Infrastructure for Operations Research*, **57**(10), 3.

Bezanson J, Edelman A, Karpinski S, Shah VB (2017). "Julia: A Fresh Approach to Numerical Computing." *SIAM Review*, **59**(1), 65–98.

Bradbury J, Frostig R, Hawkins P, Johnson MJ, Leary C, Maclaurin D, Necula G, Paszke A, VanderPlas J, Wanderman-Milne S, Zhang Q (2018). "JAX: Composable Transformations of Python+NumPy Programs." Github repository. URL https://github.com/jax-ml/jax.

Buckner J, Wilson J, Seligman M, Athey B, Watson S, Meng F (2010). "The gputools Package Enables GPU Computing in R." *Bioinformatics*, **26**(1), 134–135. doi:10.1093/bioinformatics/btp608.

Bürkner PC (2017). "brms: An R package for Bayesian multilevel models using Stan." *Journal of statistical software*, **80**, 1–28.

Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A (2017). "Stan: A probabilistic programming language." *Journal of statistical software*, **76**, 1–32.

Carpenter B, Hoffman MD, Brubaker M, Lee D, Li P, Betancourt M (2015). "The Stan Math Library: Reverse-Mode Automatic Differentiation in C++." *arXiv preprint arXiv:1509.07164*.

de Valpine P, Turek D, Paciorek CJ, Anderson-Bergman C, Lang DT, Bodik R (2017). "Programming with Models: Writing Statistical Algorithms for General Model Structures with NIMBLE." *Journal of Computational and Graphical Statistics*, **26**(2), 403–413. doi:10.1080/10618600.2016.1172487.

Degris T (2024). "GX: A Domain Specific Programming Language for Array-Computing." URL https://github.com/gx-org/gx.

Determan C (2019). *gpuR: GPU Functions for R Objects*. R package, archived on CRAN in November 2025, URL https://CRAN.R-project.org/package=gpuR.

Eddelbuettel D, François R (2011). "Rcpp: Seamless R and C++ Integration." *Journal of Statistical Software*, **40**(8), 1–18. doi:10.18637/jss.v040.i08.

Elixir Nx Team (2026). *Nx: Multi-Dimensional Arrays (Tensors) and Numerical Definitions for Elixir*. URL https://github.com/elixir-nx/nx.

EnzymeAD Contributors (2026). *Reactant.jl: Optimize Julia Functions with MLIR and XLA*. URL https://github.com/EnzymeAD/Reactant.jl.

Falbel D, Luraschi J (2025a). "Memory Management." https://torch.mlverse.org/docs/articles/memory-management. Torch for R documentation, accessed 2026-09-25.

Falbel D, Luraschi J (2025b). *torch: Tensors and Neural Networks with 'GPU' Acceleration*. doi:10.32614/CRAN.package.torch. R package version 0.17.0, URL https://CRAN.R-project.org/package=torch.

Fidler M, Hallow M, Wilkins J, Wang W (2025). *rxode2: Facilities for Simulating from ODE-Based Models*. R package version 5.0.1, URL https://CRAN.R-project.org/package=rxode2.

FitzJohn R (2025). *odin: ODE Generation and Integration*. doi:10.32614/CRAN.package.odin. R package version 1.2.7, URL https://CRAN.R-project.org/package=odin.

Gabry J, Češnovar R, Johnson A, Bronder S (2025). *cmdstanr: R Interface to 'CmdStan'*. R package version 0.9.0, URL https://mc-stan.org/cmdstanr/.

Golding N (2019). "greta: Simple and Scalable Statistical Modelling in R." *Journal of Open Source Software*, **4**(40), 1601. doi:10.21105/joss.01601.

Graham H (2026). *garry: Lazy, Spatial-Aware Raster Arrays with JIT-Compiled Kernels*. R package version 0.1.0, URL https://github.com/belian-earth/garry.

Griewank A, Walther A (2008). *Evaluating derivatives: principles and techniques of algorithmic differentiation*. 2nd edition. SIAM. doi:10.1137/1.9780898717761.

Guennebaud G, Jacob B, *et al.* (2010). ***Eigen*** *v3*. URL https://eigen.tuxfamily.org.

Hannun A, Digani J, Katharopoulos A, Collobert R (2023). "MLX: Efficient and Flexible Machine Learning on Apple Silicon." URL https://github.com/ml-explore/mlx.

Harris CR, Millman KJ, Van Der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, Berg S, Smith NJ, *et al.* (2020). "Array Programming with NumPy." *Nature*, **585**(7825), 357–362.

He H, Zou R (2021). "functorch: JAX-like Composable Function Transforms for PyTorch." URL https://github.com/pytorch/functorch.

Hess S, Palma D (2019). "Apollo: A Flexible, Powerful and Customisable Freeware Package for Choice Model Estimation and Application." *Journal of Choice Modelling*, **32**, 100170. doi:10.1016/j.jocm.2019.100170.

Kalinowski T (2026). *quickr: Compiler for R*. R package version 0.3.0.9000, URL https://github.com/t-kalinowski/quickr.

Kalinowski T, Allaire J, Chollet F (2026a). *keras3: R Interface to Keras*. R package version 1.5.1, URL https://CRAN.R-project.org/package=keras3.

Kalinowski T, Ushey K, Allaire J, Tang Y (2026b). *reticulate: Interface to 'Python'*. doi:10.32614/CRAN.package.reticulate. R package version 1.47.0, URL https://CRAN.R-project.org/package=reticulate.

Krämer K, Xie F (2026). *ast2ast*. URL https://github.com/Konrad1991/ast2ast/.

Kristensen K (2025). *RTMB: R Bindings for TMB*. doi:10.32614/CRAN.package.RTMB. URL https://CRAN.R-project.org/package=RTMB.

Kristensen K, Nielsen A, Berg CW, Skaug H, Bell BM (2016). "TMB: Automatic Differentiation and Laplace Approximation." *Journal of Statistical Software*, **70**(5). ISSN 1548-7660. doi:10.18637/jss.v070.i05. URL http://dx.doi.org/10.18637/jss.v070.i05.

Lobato-Fernández C, Ferrer-Bonsoms JA, Rubio A (2025). *GPUmatrix: Basic Linear Algebra with GPU*. R package version 1.0.3, URL https://CRAN.R-project.org/package=GPUmatrix.

Maclaurin D, Duvenaud D, Adams RP (2015). "Autograd: Effortless Gradients in NumPy." In *ICML 2015 AutoML Workshop*. URL https://github.com/HIPS/autograd.

Morandat F, Hill B, Osvald L, Vitek J (2012). "Evaluating the design of the R language: Objects and functions for data analysis." In *European conference on object-oriented programming*, pp. 104–131. Springer.

Neal RM (2011). "MCMC Using Hamiltonian Dynamics." In S Brooks, A Gelman, GL Jones, XL Meng (eds.), *Handbook of Markov Chain Monte Carlo*, pp. 113–162. Chapman and Hall/CRC.

Nocedal J, Wright SJ (2006). *Numerical Optimization*. 2nd edition. Springer-Verlag, New York. doi:10.1007/978-0-387-40065-5.

OpenXLA Team (2026). *OpenXLA*. URL https://openxla.org/.

Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, *et al.* (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in neural information processing systems*, **32**.

Pav SE (2023). *madness: Automatic Differentiation of Multivariate Operations*. R package version 0.2.8, URL https://github.com/shabbychef/madness.

Perdry H (2024). *salad: Simple Automatic Differentiation*. doi:10.32614/CRAN.package.salad. R package version 1.2, URL https://CRAN.R-project.org/package=salad.

Pfeifer J (2026a). *GoMLX: An Accelerated Machine Learning Framework for Go*. URL https://github.com/gomlx/gomlx.

Pfeifer J (2026b). *gopjrt: Go Wrappers for OpenXLA PJRT*. URL https://github.com/gomlx/gopjrt.

PJRT Team (2026). *PJRT: Pluggable JIT RunTime*. URL https://openxla.org/pjrt/.

Posit Software, PBC (2026a). "Arrays in R and Python." https://rstudio.github.io/reticulate/articles/arrays.html. Reticulate vignette, accessed 2026-09-24.

Posit Software, PBC (2026b). "Declare Python Requirements: py_require()." https://rstudio.github.io/reticulate/reference/py_require.html. Reticulate 1.47.0 reference, accessed 2026-09-24.

Posit Software, PBC (2026c). "Introduction to Tensor Slicing." https://tensorflow.rstudio.com/guides/tensorflow/tensor_slicing. TensorFlow for R guide, accessed 2026-09-24.

R Core Team (2025a). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

R Core Team (2025b). *Writing R Extensions*. R Foundation for Statistical Computing, Vienna, Austria. URL https://CRAN.R-project.org/doc/manuals/r-release/R-exts.html.

Sartore L (2025). *dual: Automatic Differentiation with Dual Numbers*. doi:10.32614/CRAN.package.dual. R package version 0.0.6, URL https://CRAN.R-project.org/package=dual.

StableHLO Team (2026). *StableHLO*. URL https://openxla.org/stablehlo/spec.

The IREE Authors (2019). "IREE." URL https://github.com/iree-org/iree.

The JAX Authors (2024). "Autodidax: JAX Core from Scratch." URL https://docs.jax.dev/en/latest/autodidax.html.

The MicroJAX Authors (2024). *MicroJAX*. URL https://github.com/joey00072/microjax.

Tierney L, Becker G, Kalibera T (2018). "ALTREP: Alternative Representations for R Objects." URL https://svn.r-project.org/R/branches/ALTREP/ALTREP.html.

Towell A (2026). *nabla: Exact Derivatives via Automatic Differentiation*. R package version 0.7.1, URL https://github.com/queelius/nabla.

Turcotte A, Goel A, Křikava F, Vitek J (2020). "Designing Types for R, Empirically." *Proceedings of the ACM on Programming Languages*, **4**(OOPSLA), 181:1–181:25. doi:10.1145/3428249.

Urbanek S (2026). *OpenCL: Interface Allowing R to Use OpenCL*. R package version 0.2-11, URL https://CRAN.R-project.org/package=OpenCL.

Wang J, Morgan M (2025). *gpuMagic: An OpenCL Compiler with the Capacity to Compile R Functions and Run the Code on GPU*. Bioconductor package, removed with Bioconductor 3.23, URL https://bioconductor.org/packages/gpuMagic.

Wilke CO, Thomason A, Reimert MM, Kosenkov I, Barrett M, Parry J, Vernon K (2026). *rextendr: Build 'Rust' Powered 'R' Packages*. R package version 0.5.0, URL https://CRAN.R-project.org/package=rextendr.

A. Pitfalls of base R's array interface

Base R's array interface grew around vectors, and several of its rules give results that depend on the length of an index or argument rather than on its meaning. In interactive use, such results are noticed and corrected, but in a compiled program, whose shapes are fixed, they would become hard-to-find bugs or prevent compilation altogether. Table 3 lists the cases we encountered most often and how **anvl** handles them; the code for each case is part of the replication material.

Operation	Base R	**anvl**
`m[i, ]` with `length(i) == 1`	drops the axis unless `drop = FALSE`	a scalar index drops the axis, an array index keeps it, whatever its length
`x[-i]` with `i = integer(0)`	returns an empty vector instead of `x`	negative indices are not supported; masks select elements
`x + y` with different lengths	recycles the shorter vector, warning only if the lengths do not divide	an error; only scalars are broadcast
`m + r` for a 2×3 matrix `m` and a 1×3 matrix `r`	an error; requires `sweep()`	explicit broadcasting via `nv_broadcast_arrays()`
`cumsum()`, `sort()`, `rev()` of a matrix	flatten the matrix	operate along a given axis and keep the shape
reduction over the second axis of a 3-d array	requires `apply()`, which loops in R	`nv_sum(x, axes = 2)` for any set of axes
`A %*% B` for 3-d arrays `A`, `B`	silently returns the inner product of the flattened arrays	batched matrix product along the leading axes
`v %*% v` for a vector `v`	a 1×1 matrix; vectors silently become rows or columns	an error; operands need at least two axes
integer overflow, other data types	`NA` with a warning; no single precision, 64-bit, or unsigned integers	explicit data types; values that a data type cannot hold are an error
`sample(x)` with `length(x) == 1`	samples from `1:x`	`nv_sample()` always samples from the given array

Table 3: Operations of base R's array interface that are hard to express or give surprising results, and their counterparts in **anvl**.

Package	Double `NA`	Integer `NA`	`NA_integer_ + 1L`	Logical `NA`
anvl	kept	kept (warning)	-2147483647	error
torch	kept	kept	-2147483647	TRUE
tensorflow	kept	kept	-2147483647	TRUE
greta	error	error	–	error
quickr	kept	kept	-2147483647	TRUE
ast2ast	kept	kept	NA	kept
Stan	error	error	–	error
TMB	kept	0	–	0
RTMB	kept	kept	–	kept
nimble	kept	kept	-2147483647	TRUE

Table 4: Missing values after a round trip through a minimal program of each package. kept: R recognizes the result as missing again; error: the package refuses the value; otherwise the value the missing value turned into. The integer column of **RTMB** uses doubles, as its tapes only support doubles.

B. Missing values

R represents missing values by special values of its data types: `NA_real_` is a NaN with a particular payload, `NA_integer_` is the smallest 32-bit integer, and a logical `NA` has no counterpart in a boolean type. Array libraries and compilers that are not specific to R do not know these conventions, so missing values can change their meaning when data enters or leaves a program. Table 4 shows what the packages compared in Table 1 return for R's missing values after passing them through a minimal program, and whether `NA_integer_ + 1L` still yields `NA`, as in R. The table is generated by the script `verification/missing_values.R` in the replication material.

For doubles, most packages return `NA` unchanged, as the NaN payload happens to survive the computation, while **greta** and Stan refuse missing values. For integers, only **ast2ast** treats `NA_integer_` as missing in arithmetic; in the other packages, including **anvl**, it is an ordinary integer inside the program, and `NA_integer_ + 1L` silently yields $-2^{31} + 1$, of which **anvl** only warns when the value returns to R. A logical `NA`, finally, silently becomes `TRUE` or 0 in most packages, whereas **anvl** refuses it.

Affiliation:

Sebastian Fischer
Department of Statistics
Ludwig-Maximilians-Universität München
Ludwigstr. 33, 80539 Munich, Germany
E-mail: sebastian.fischer@stat.uni-muenchen.de

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd